Backend Development Plan

Startup Investment Readiness & Deal Intelligence Platform

Executive Summary

This backend plan bridges the **current MVP (3 APIs)** with the **full PRD vision (6 core features + platform capabilities)**. We'll build upon the existing FastAPI foundation while architecting for scalability, multi-tenancy (VC-specific instances), and AI-driven intelligence.

---

1. Current State Analysis

Existing APIs (MVP - Phase 1 Complete)

✅ **Pitch Deck Analysis** - `/api/pitch-deck/analyze`
✅ **Investment Readiness** - `/api/investment/readiness`
✅ **Valuation Benchmarking** - `/api/valuation/benchmark`

Technology Stack (Current)

Framework: FastAPI (Python)
Deployment: Local development (localhost:8000)
Storage: File-based (logs)
AI: Likely LLM-based (OpenAI/Claude/Gemini)
Authentication: None (development mode)

---

2. Gap Analysis: MVP → Full Product

Feature	Current Status	PRD Requirement	Priority
Pitch Deck Analyzer	✅ Functional	Enhanced with percentile ranking	P0
Investment Readiness	✅ Functional	Add temporal tracking	P0
Valuation Benchmarking	✅ Functional	Add confidence bands & comparables	P1
Investor Matching	❌ Missing	Build from scratch	P1

Feature	Current Status	PRD Requirement	Priority
Founder Circles	❌ Missing	Community platform	P2
Founder Dashboard/CRM	❌ Missing	Tracking & analytics	P1
Multi-tenancy	❌ Missing	VC-specific isolation	P0
Authentication	❌ Missing	OAuth/API keys	P0
Database	❌ Missing	PostgreSQL + vector DB	P0
Real-time updates	❌ Missing	WebSocket support	P2

---

3. Backend Architecture

3.1 System Architecture Diagram

```
┌─────────────
```

3.2 Microservices Architecture

```
services/
├── api_gateway/        # FastAPI main app
├── ai_services/        # LLM orchestration
│   ├── pitch_analyzer/
│   ├── readiness_scorer/
│   └── valuation_engine/
├── business_logic/
│   ├── investor_matching/
│   ├── user_management/
│   └── analytics/
├── data_services/
│   ├── file_storage/
│   ├── database/
│   └── cache/
└── integrations/
    ├── crunchbase/
    ├── tracxn/
    └── notifications/
```

---

4. Database Schema Design

4.1 Core Tables

```
-- Multi-tenancy foundation
CREATE TABLE organizations (
    org_id UUID PRIMARY KEY,
    org_name VARCHAR(255) NOT NULL,
    org_type VARCHAR(50), -- 'VC', 'ACCELERATOR', 'ANGEL_NETWORK'
    subscription_tier VARCHAR(50),
    settings JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Users (VCs and Founders)
CREATE TABLE users (
    user_id UUID PRIMARY KEY,
    org_id UUID REFERENCES organizations(org_id),
    email VARCHAR(255) UNIQUE NOT NULL,
    user_type VARCHAR(50), -- 'VC_ANALYST', 'VC_PARTNER', 'FOUNDER'
    profile_data JSONB,
    created_at TIMESTAMP DEFAULT NOW(),
    last_login TIMESTAMP
);
```

```
-- Startups
CREATE TABLE startups (
    startup_id UUID PRIMARY KEY,
    founder_id UUID REFERENCES users(user_id),
    org_id UUID REFERENCES organizations(org_id), -- VC they're associated with
    startup_name VARCHAR(255) NOT NULL,
    sector VARCHAR(100),
    stage VARCHAR(50),
    founded_year INTEGER,
    location VARCHAR(100),
    team_size INTEGER,
    metadata JSONB, -- revenue, growth_rate, customers, etc.
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Pitch Deck Analysis
CREATE TABLE pitch_analyses (
    analysis_id UUID PRIMARY KEY,
    startup_id UUID REFERENCES startups(startup_id),
    deck_file_url VARCHAR(500),
    deck_version INTEGER DEFAULT 1,
    overall_score FLOAT,
    idea_score FLOAT,
    team_score FLOAT,
    market_score FLOAT,
    traction_score FLOAT,
    risk_score FLOAT,
    scalability_score FLOAT,
    flags JSONB, -- red, amber, green flags
    suggestions TEXT[],
    percentile_rank FLOAT, -- vs similar startups
    ai_model_used VARCHAR(50),
    analysis_metadata JSONB,
    analyzed_at TIMESTAMP DEFAULT NOW()
);

-- Investment Readiness Tracking
CREATE TABLE readiness_scores (
    score_id UUID PRIMARY KEY,
    startup_id UUID REFERENCES startups(startup_id),
    overall_score FLOAT,
    pitch_quality_score FLOAT,
    traction_score FLOAT,
    team_score FLOAT,
    market_timing_score FLOAT,
    financial_health_score FLOAT,
    readiness_level VARCHAR(50), -- 'READY', 'NEARLY_READY', 'NOT_READY'
    improvements_needed JSONB,
    time_to_readiness_days INTEGER,
```

```
    scored_at TIMESTAMP DEFAULT NOW()
);

-- Valuation Benchmarks
CREATE TABLE valuations (
    valuation_id UUID PRIMARY KEY,
    startup_id UUID REFERENCES startups(startup_id),
    low_estimate DECIMAL(15,2),
    median_estimate DECIMAL(15,2),
    high_estimate DECIMAL(15,2),
    confidence_level VARCHAR(50),
    methodology VARCHAR(100),
    comparable_companies JSONB,
    benchmarks_used JSONB,
    calculated_at TIMESTAMP DEFAULT NOW()
);

-- Investor Database
CREATE TABLE investors (
    investor_id UUID PRIMARY KEY,
    org_id UUID REFERENCES organizations(org_id),
    investor_name VARCHAR(255),
    investor_type VARCHAR(50), -- 'ANGEL', 'MICRO_VC', 'VC_FUND'
    sectors TEXT[],
    stages TEXT[],
    geographies TEXT[],
    ticket_size_min DECIMAL(15,2),
    ticket_size_max DECIMAL(15,2),
    portfolio_companies JSONB,
    contact_info JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Investor Matches
CREATE TABLE investor_matches (
    match_id UUID PRIMARY KEY,
    startup_id UUID REFERENCES startups(startup_id),
    investor_id UUID REFERENCES investors(investor_id),
    match_score FLOAT,
    match_reasoning JSONB,
    match_status VARCHAR(50), -- 'SUGGESTED', 'CONTACTED', 'MEETING', 'DECLINED'
    created_at TIMESTAMP DEFAULT NOW()
);

-- Founder Communities
CREATE TABLE communities (
    community_id UUID PRIMARY KEY,
    org_id UUID REFERENCES organizations(org_id),
    community_name VARCHAR(255),
    community_type VARCHAR(50), -- 'STAGE', 'SECTOR', 'GEOGRAPHY'
```

```
    criteria JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE community_members (
    membership_id UUID PRIMARY KEY,
    community_id UUID REFERENCES communities(community_id),
    user_id UUID REFERENCES users(user_id),
    joined_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE community_posts (
    post_id UUID PRIMARY KEY,
    community_id UUID REFERENCES communities(community_id),
    author_id UUID REFERENCES users(user_id),
    content TEXT,
    post_type VARCHAR(50), -- 'DISCUSSION', 'DECK_REVIEW', 'QUESTION'
    attachments JSONB,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Fundraising CRM
CREATE TABLE fundraising_activities (
    activity_id UUID PRIMARY KEY,
    startup_id UUID REFERENCES startups(startup_id),
    investor_id UUID REFERENCES investors(investor_id) NULL,
    activity_type VARCHAR(100), -- 'EMAIL_SENT', 'MEETING', 'FOLLOW_UP', 'PITCH'
    notes TEXT,
    next_action VARCHAR(255),
    next_action_date DATE,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Analytics & Metrics
CREATE TABLE platform_metrics (
    metric_id UUID PRIMARY KEY,
    org_id UUID REFERENCES organizations(org_id),
    metric_type VARCHAR(100),
    metric_value JSONB,
    recorded_at TIMESTAMP DEFAULT NOW()
);
```

4.2 Vector Database Schema (Pinecone/Weaviate)

```
 # For semantic search and investor matching
vector_schema = {
    "startup_embeddings": {
        "id": "startup_id",
        "vector": [1536],  # OpenAI embedding dimension
```

```
        "metadata": {
            "sector": str,
            "stage": str,
            "traction_level": str,
            "team_strength": float,
            "pitch_summary": str
        }
    },
    "investor_embeddings": {
        "id": "investor_id",
        "vector": [1536],
        "metadata": {
            "sectors": list,
            "stages": list,
            "investment_thesis": str,
            "portfolio_focus": str
        }
    }
}
```

---

5. API Architecture Enhancement

5.1 Authentication & Authorization

```
 # JWT-based authentication
from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

class AuthService:
    def authenticate_user(email: str, password: str):
        # Validate credentials
        # Return JWT token
        pass

    def verify_token(token: str = Depends(oauth2_scheme)):
        # Decode JWT
        # Check org_id, user_id, permissions
        pass

    def check_org_access(user: User, startup_id: UUID):
        # Verify user belongs to org that owns startup
        pass
```

5.2 Enhanced API Endpoints

Phase 1 Enhancements (Existing APIs)

```
 # Enhanced Pitch Deck Analysis
@app.post("/api/v1/pitch-deck/analyze")
async def analyze_pitch_deck(
    file: UploadFile,
    startup_id: UUID,
    version: int = 1,
    user: User = Depends(get_current_user)
):
    """
    Enhancements:
    - Store in PostgreSQL + S3
    - Calculate percentile rank
    - Version tracking
    - Historical comparison
    """
    pass

# Enhanced Investment Readiness
@app.post("/api/v1/investment/readiness")
async def calculate_readiness(
    startup_id: UUID,
    user: User = Depends(get_current_user)
):
    """
    Enhancements:
    - Pull from startup profile
    - Track score over time
    - Generate improvement roadmap
    """
    pass

# Enhanced Valuation Benchmarking
@app.post("/api/v1/valuation/benchmark")
async def benchmark_valuation(
    startup_id: UUID,
    user: User = Depends(get_current_user)
):
    """
    Enhancements:
    - Fetch real comparable companies
    - Confidence intervals
    - Market trend analysis
    """
    pass
```

Phase 2: New Features

```
 # Investor Matching
@app.post("/api/v1/investor-matching/match")
async def match_investors(
    startup_id: UUID,
    limit: int = 20,
    user: User = Depends(get_current_user)
):
    """
    Returns:
    - Ranked investor list
    - Match scores & reasoning
    - Contact recommendations
    """
    pass

@app.get("/api/v1/investor-matching/investors")
async def list_investors(
    sector: str = None,
    stage: str = None,
    geography: str = None,
    user: User = Depends(get_current_user)
):
    pass

# Founder Dashboard
@app.get("/api/v1/dashboard/overview")
async def get_dashboard(
    startup_id: UUID,
    user: User = Depends(get_current_user)
):
    """
    Returns:
    - Current readiness score
    - Score trend (last 6 months)
    - Deck version history
    - Investor outreach status
    - Upcoming actions
    """
    pass

# Fundraising CRM
@app.post("/api/v1/crm/activities")
async def log_activity(
    activity: ActivityCreate,
    user: User = Depends(get_current_user)
):
    pass

@app.get("/api/v1/crm/pipeline")
```

```
async def get_pipeline(
    startup_id: UUID,
    user: User = Depends(get_current_user)
):
    """
    Returns fundraising pipeline visualization data
    """
    pass

# Communities
@app.get("/api/v1/communities")
async def list_communities(user: User = Depends(get_current_user)):
    pass

@app.post("/api/v1/communities/{community_id}/posts")
async def create_post(
    community_id: UUID,
    post: PostCreate,
    user: User = Depends(get_current_user)
):
    pass

# Analytics (VC Admin)
@app.get("/api/v1/analytics/deal-flow")
async def get_deal_flow_analytics(
    org_id: UUID,
    start_date: date,
    end_date: date,
    user: User = Depends(get_current_user)
):
    """
    Only accessible by VC admins
    Returns deal screening metrics
    """
    pass
```

---

6. AI Services Architecture

6.1 LLM Orchestration Layer

```
class LLMOrchestrator:
    """
    Model-agnostic LLM interface
    Supports OpenAI, Claude, Gemini
    """

    def __init__(self, provider: str = "openai"):
```

```
        self.provider = provider
        self.client = self._initialize_client()

    async def analyze_pitch_deck(self, text: str, metadata: dict):
        """
        Hybrid approach:
        1. Rule-based extraction
        2. LLM-based analysis
        3. Scoring normalization
        """
        prompt = self._build_pitch_analysis_prompt(text, metadata)
        response = await self.client.complete(prompt)
        return self._parse_structured_response(response)

    async def calculate_readiness(self, startup_data: dict):
        """
        Weighted scoring with LLM qualitative assessment
        """
        pass

    async def generate_investor_matches(self, startup_profile, investor_pool):
        """
        Semantic matching + rule-based filtering
        """
        embeddings = await self._generate_embeddings(startup_profile)
        similar_investors = vector_db.search(embeddings, top_k=50)
        ranked = self._apply_business_rules(similar_investors, startup_profile)
        return ranked
```

6.2 Prompt Engineering Framework

```
 # Structured prompts with JSON output
PITCH_ANALYSIS_PROMPT = """
You are an expert VC analyst evaluating a startup pitch deck.

Analyze the following pitch deck content and provide scores (0-100) for:
1. Idea Novelty & Market Fit
2. Team Capability
3. Market Opportunity
4. Traction Feasibility
5. Risk & Sustainability
6. Scalability Potential

Context:
- Sector: {sector}
- Stage: {stage}
- Team Size: {team_size}

Pitch Deck Content:
```

```
{deck_text}

Respond ONLY with valid JSON in this format:
{{
  "overall_score": 0-100,
  "scores": {{
    "idea": 0-100,
    "team": 0-100,
    "market": 0-100,
    "traction": 0-100,
    "risk": 0-100,
    "scalability": 0-100
  }},
  "flags": {{
    "red": ["flag 1", "flag 2"],
    "amber": ["flag 1"],
    "green": ["flag 1", "flag 2"]
  }},
  "suggestions": ["improvement 1", "improvement 2"],
  "summary": "2-3 sentence summary"
}}
"""
```

---

7. Data Integration Layer

7.1 External Data Sources

```
class BenchmarkDataService:
    """
    Integrate with Crunchbase, Tracxn, etc.
    """

    async def fetch_comparable_companies(
        self,
        sector: str,
        stage: str,
        geography: str = "India"
    ):
        """
        Returns companies with:
        - Valuation data
        - Funding history
        - Metrics (revenue, growth)
        """
        pass

    async def get_sector_benchmarks(self, sector: str):
```

```
        """
        Revenue multiples, growth rates, etc.
        """
        pass

    async def get_investor_portfolio(self, investor_id: UUID):
        """
        Pull portfolio companies from public databases
        """
        pass
```

7.2 File Processing Pipeline

```
class FileProcessor:
    """
    Extract text and structure from pitch decks
    """

    async def process_pitch_deck(self, file_path: str, file_type: str):
        if file_type == 'pptx':
            return await self._process_pptx(file_path)
        elif file_type == 'pdf':
            return await self._process_pdf(file_path)

    async def _process_pptx(self, file_path):
        # Use python-pptx
        # Extract text, images, charts
        # Maintain slide structure
        pass

    async def _process_pdf(self, file_path):
        # Use PyPDF2 or pdfplumber
        # OCR if needed
        pass

    async def generate_embeddings(self, text: str):
        # OpenAI embeddings API
        # Cache in vector DB
        pass
```

---

8. Security & Privacy Implementation

8.1 Multi-Tenancy Isolation

```
class OrgIsolationMiddleware:
    """
```

```
    Ensure data access is scoped to organization
    """

    async def __call__(self, request, call_next):
        user = request.state.user
        org_id = user.org_id

        # Inject org_id into query context
        request.state.org_id = org_id

        response = await call_next(request)
        return response

# Database queries always filtered by org_id
def get_startups(org_id: UUID):
    return db.query(Startup).filter(Startup.org_id == org_id).all()
```

8.2 Data Encryption

```
# At-rest encryption (PostgreSQL)
# - Transparent Data Encryption (TDE)
# - Field-level encryption for sensitive data

# In-transit encryption
# - TLS 1.3 for all API communication
# - Encrypted S3 uploads

# Secrets management
# - AWS Secrets Manager / HashiCorp Vault
# - No hardcoded credentials
```

8.3 Access Control

```
class PermissionService:
    PERMISSIONS = {
        'VC_ADMIN': ['*'],
        'VC_ANALYST': ['read:all', 'write:analysis'],
        'FOUNDER': ['read:own', 'write:own'],
        'LIMITED_FOUNDER': ['read:own']
    }

    def check_permission(self, user: User, action: str, resource: str):
        # Role-based access control
        pass
```

---

9. Performance & Scalability

9.1 Caching Strategy

```
# Redis caching layers
CACHE_CONFIG = {
    'startup_profiles': 3600,  # 1 hour
    'investor_data': 86400,    # 24 hours
    'benchmark_data': 604800,  # 7 days
    'analysis_results': 2592000  # 30 days
}

class CacheService:
    async def get_or_compute(self, key, compute_fn, ttl):
        cached = await redis.get(key)
        if cached:
            return json.loads(cached)

        result = await compute_fn()
        await redis.setex(key, ttl, json.dumps(result))
        return result
```

9.2 Async Processing

```
from celery import Celery

# Background tasks for heavy operations
celery_app = Celery('startup_platform', broker='redis://localhost')

@celery_app.task
def analyze_pitch_deck_async(analysis_id: UUID):
    # Process deck in background
    # Update database when complete
    # Send notification to user
    pass

@celery_app.task
def generate_valuation_report_async(startup_id: UUID):
    # Fetch comparable companies
    # Run calculations
    # Generate PDF report
    pass
```

9.3 Database Optimization

```
-- Indexes for common queries
CREATE INDEX idx_startups_org_sector ON startups(org_id, sector);
CREATE INDEX idx_startups_stage ON startups(stage);
CREATE INDEX idx_pitch_analyses_startup ON pitch_analyses(startup_id, analyzed_at DESC);
CREATE INDEX idx_readiness_scores_startup ON readiness_scores(startup_id, scored_at DESC);
CREATE INDEX idx_investors_sector_stage ON investors USING GIN(sectors, stages);

-- Partitioning for large tables
CREATE TABLE pitch_analyses_2026 PARTITION OF pitch_analyses
FOR VALUES FROM ('2026-01-01') TO ('2027-01-01');
```

---

10. Monitoring & Observability

10.1 Logging

```
import structlog

logger = structlog.get_logger()

logger.info(
    "pitch_deck_analyzed",
    startup_id=startup_id,
    overall_score=score,
    processing_time_ms=duration,
    ai_model=model_name
)
```

10.2 Metrics

```
from prometheus_client import Counter, Histogram

# Business metrics
pitch_analyses = Counter('pitch_analyses_total', 'Total pitch deck analyses')
readiness_scores = Histogram('readiness_score_distribution', 'Distribution of readiness scores')

# Performance metrics
api_latency = Histogram('api_request_duration_seconds', 'API request latency')
llm_latency = Histogram('llm_request_duration_seconds', 'LLM API latency')
```

10.3 Error Tracking

```
import sentry_sdk

sentry_sdk.init(
```

```
    dsn="...",
    traces_sample_rate=0.1,
    environment="production"
)
```

---

11. Testing Strategy

11.1 Unit Tests

```
 # tests/test_pitch_analyzer.py
import pytest
from services.ai_services.pitch_analyzer import PitchAnalyzer

@pytest.mark.asyncio
async def test_pitch_analysis_scoring():
    analyzer = PitchAnalyzer()
    result = await analyzer.analyze(mock_deck_text, metadata)

    assert 0 <= result.overall_score <= 100
    assert len(result.suggestions) > 0
    assert 'idea' in result.scores
```

11.2 Integration Tests

```
 # tests/integration/test_analysis_workflow.py
async def test_end_to_end_analysis():
    # Upload deck
    # Process analysis
    # Calculate readiness
    # Generate valuation
    # Verify database state
    pass
```

11.3 Load Tests

```
 # locustfile.py
from locust import HttpUser, task

class StartupPlatformUser(HttpUser):
    @task
    def analyze_deck(self):
        self.client.post("/api/v1/pitch-deck/analyze", files=...)
```

---

12. Deployment Architecture

12.1 Infrastructure (AWS)

```
# Terraform configuration
resource "aws_ecs_cluster" "platform" {
  name = "startup-platform-cluster"
}

resource "aws_rds_instance" "postgres" {
  engine = "postgres"
  instance_class = "db.t3.large"
  multi_az = true
  backup_retention_period = 30
}

resource "aws_elasticache_cluster" "redis" {
  engine = "redis"
  node_type = "cache.t3.medium"
}

resource "aws_s3_bucket" "pitch_decks" {
  bucket = "startup-platform-pitch-decks"
  versioning {
    enabled = true
  }
  encryption {
    enabled = true
  }
}
```

12.2 CI/CD Pipeline

```
# .github/workflows/deploy.yml
name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Run tests
        run: pytest
```

```
    - name: Lint code
      run: black --check .

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to ECS
        run: |
          aws ecs update-service \
            --cluster startup-platform \
            --service api \
            --force-new-deployment
```

---

13. Phase-wise Implementation Roadmap

Phase 1: Foundation (Weeks 1-4)

- [ ] Set up PostgreSQL schema
- [ ] Implement authentication & multi-tenancy
- [ ] Migrate existing APIs to new architecture
- [ ] Set up S3 file storage
- [ ] Build user & organization management

Phase 2: Core Features Enhancement (Weeks 5-8)

- [ ] Enhanced pitch deck analysis with percentile ranking
- [ ] Temporal readiness score tracking
- [ ] Valuation with confidence intervals
- [ ] Vector database setup for matching
- [ ] Founder dashboard MVP

Phase 3: Investor Matching (Weeks 9-12)

- [ ] Investor database & profiles
- [ ] Semantic matching engine
- [ ] Match scoring algorithm
- [ ] Investor search & filters
- [ ] Match notification system

Phase 4: CRM & Communities (Weeks 13-16)

- [ ] Fundraising activity tracking

- [ ] Pipeline visualization
- [ ] Community platform
- [ ] Deck sharing & feedback
- [ ] Real-time updates (WebSocket)

Phase 5: Analytics & Optimization (Weeks 17-20)

- [ ] VC analytics dashboard
- [ ] Deal flow metrics
- [ ] Performance optimization
- [ ] Advanced caching
- [ ] Load testing & scaling

---

14. API Documentation

14.1 OpenAPI Spec Enhancement

```
 from fastapi import FastAPI
from fastapi.openapi.utils import get_openapi

def custom_openapi():
    if app.openapi_schema:
        return app.openapi_schema

    openapi_schema = get_openapi(
        title="Startup Investment Platform API",
        version="2.0.0",
        description="""
        AI-powered platform for startup evaluation and VC deal flow.

        ## Features
        - Pitch deck analysis with AI
        - Investment readiness scoring
        - Valuation benchmarking
        - Investor matching
        - Fundraising CRM

        ## Authentication
        Use JWT bearer tokens in the Authorization header.
        """,
        routes=app.routes,
    )

    app.openapi_schema = openapi_schema
    return app.openapi_schema
```

```
app.openapi = custom_openapi
```

---

15. Success Metrics

Technical KPIs

API response time p99 < 2 seconds
Pitch deck analysis < 3 minutes
System uptime > 99.9%
Zero data breaches
Support 800+ concurrent users

Business KPIs

Reduction in VC screening time by 60%
80% accuracy in investor matching
90% founder satisfaction with feedback quality
50% improvement in founder readiness scores over 3 months

---

16. Risk Mitigation

Risk	Impact	Mitigation
AI inconsistency	High	Hybrid scoring, validation rules, human review
Data privacy breach	Critical	Encryption, audit logs, penetration testing
Vendor lock-in (LLM)	Medium	Model-agnostic architecture, fallback providers
Scalability limits	Medium	Auto-scaling, caching, async processing
Integration failures	Low	Circuit breakers, retry logic, monitoring

---

17. Handoff & Documentation

Developer Onboarding

README with setup instructions
Architecture decision records (ADRs)

API documentation (Swagger UI)
Database schema diagrams
Deployment runbooks

Operations Runbooks

Incident response procedures
Backup & recovery process
Scaling guidelines
Monitoring dashboards

Conclusion

This backend plan transforms the MVP into an enterprise-grade, multi-tenant platform while maintaining the core AI-driven value proposition. The architecture is designed for:

Scalability - Microservices, caching, async processing
Security - Multi-tenancy, encryption, access control
Flexibility - Model-agnostic AI, modular design
Maintainability - Clear separation of concerns, comprehensive testing

Next Steps:

Review and approve architecture
Set up development environment
Begin Phase 1 implementation
Establish weekly sprint reviews